

NYCU-ECE DCS-2026

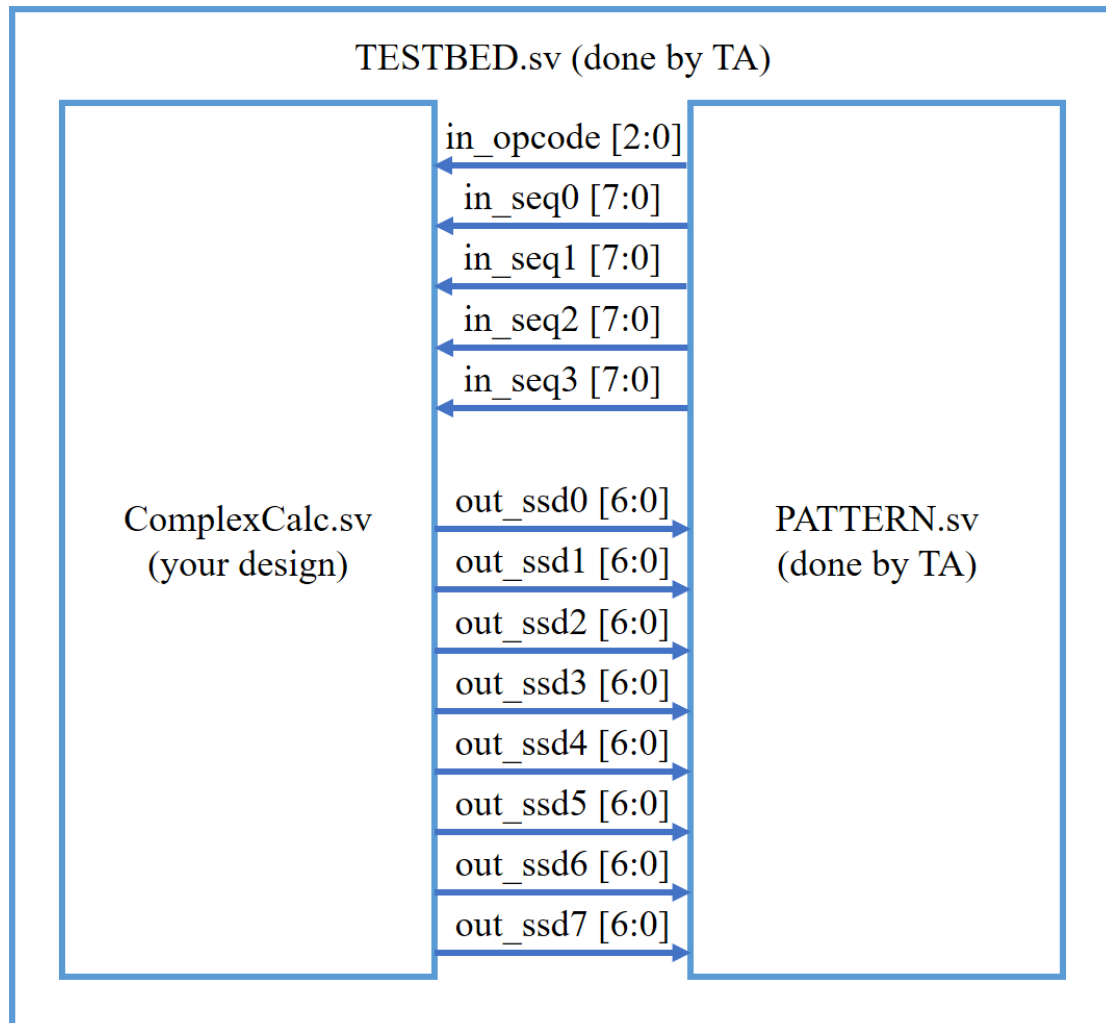
HW01

Design: Complex Number Calculation

Data Preparation

1. 從 TA 目錄資料夾解壓縮:
 `% tar -xvf ~es26dcsTA01/HW01.tar`
2. 解壓縮資料夾包含以下:
 - a. 00_TESTBED/
 - b. 01_RTL/
 - c. 02_SYN/
 - d. 03_GATE/
 - e. 09_UPLOAD/

Block Diagram



Design Description

本次作業將設計複數計算電路，詳細步驟如下：

- 1) BCD 輸入判別
- 2) 複數 Norm 計算
- 3) 硬體近似偵錯
- 4) 穩定排序演算法
- 5) 複數運算
- 6) 七段顯示器輸出

第一步，輸入訊號 in_seq0 、 in_seq1 、 in_seq2 、 in_seq3 會以 BCD 的形式表示 $a + bi$ 、 $c + di$ 、 $e + fi$ 、 $g + hi$ 。(在 BCD 形式下 $a \sim h$ 的合理範圍為 $0 \sim 9$)

Ex1. $\text{in_seq0} [7:4] = 1001$ 、 $\text{in_seq0} [3:0] = 0101$

$a = 9$ 、 $b = 5$

$\text{in_seq0} \Rightarrow 9 + 5i$

若出現非 BCD 表示的數值，或出現 $(0, 0)$ ，則需輸出 Error (詳見第七步)。

Ex2. $\text{in_seq1} [7:4] = 1010$ 、 $\text{in_seq1} [3:0] = 1101$

$c = 10$ 、 $d = 13$

$\text{in_seq1} \Rightarrow$ 錯誤! 非 BCD 表示值

Ex3. $\text{in_seq2} [7:4] = 0000$ 、 $\text{in_seq2} [3:0] = 0000$

$e = 0$ 、 $f = 0$

$\text{in_seq2} \Rightarrow$ 錯誤! 出現 $(0, 0)$

第二步，計算四個複數的 Norm。Norm 的定義是該複數在複數平面上對應的點 (x, y) 到原點 (0, 0) 的歐幾里得距離，公式表示如下：

$$\|x + yi\| = \sqrt{x^2 + y^2}$$

根號在硬體上沒有直接的實現方式，所以我們必須使用近似解處理，常見的牛頓-拉弗森方法 (Newton-Raphson method) 就是其中一個解法。假設欲開根號的數值 $x^2 + y^2 = \text{val}$ ，我們使用 $x_0 = 6$ 來做兩次遞迴迭代：

$$x_{n+1} = \frac{1}{2} \left(x_n + \frac{\text{val}}{x_n} \right)$$

第一次迭代：

$$x_1 = \frac{1}{2} \left(6 + \frac{\text{val}}{6} \right)$$

第二次迭代：

$$\begin{aligned} x_2 &= \frac{1}{2} \left[\frac{1}{2} \left(6 + \frac{\text{val}}{6} \right) + \frac{\text{val}}{\frac{1}{2} \left(6 + \frac{\text{val}}{6} \right)} \right] \\ &= \frac{\text{val}^2 + 216 * \text{val} + 1296}{24 * \text{val} + 864} \end{aligned}$$

第三步，由於牛頓-拉弗森方法會在實際根距離猜測值 x_0 較遠時產生近似偏差，我們可以引入偵錯方法來修正結果值。其中一個解法是將預測結果作平方計算，若大於 val 值，則將預測值修正 -1。(此作法偏差值必不超過 +1)

Ex4. 計算 $\|9 + 9i\|$

$\text{val} = 9^2 + 9^2 = 162$ 、實際根號: 12、牛頓-拉弗森預測結果: 13

檢測 $13^2 > 9^2 + 9^2$ 成立

修正近似值 $13 - 1 = 12$

註: 本題目複數 Norm 計算皆以截斷 (truncate) 後之結果為準，無須採用完整精度 (full precision)。PATTERN 檢測值依標準流程產生；若使用替代演算法，最終輸出結果仍必須與標準流程完全一致。

第四步，依 `in_opcode [2]` 指示，將 `in_seq0`、`in_seq1`、`in_seq2`、`in_seq3` 進行穩定排序 (stable sort)。排序的判斷條件是: Norm 值小於 7 的優先。

穩定排序 (stable sort) 的定義是: 在排序過程中，若兩個元素的排序鍵值 (key) 相同，排序完成後它們的相對順序必須與排序前保持一致。例如 Ex5 中的 `in_seq1` 和 `in_seq3`，他們因為同時滿足 Norm 值小於 7，所以在排序前後相對位置保持不變。

Ex5. 欲排序的輸入如下

$$\text{in_seq0: } \| 7 + 2i \| = 7$$

$$\text{in_seq1: } \| 3 + 4i \| = 5$$

$$\text{in_seq2: } \| 9 + 9i \| = 12$$

$$\text{in_seq3: } \| 2 + 0i \| = 2$$

Case1: `in_opcode [2] = 1'b0` => 不排序

$$\text{sorted [0] = in_seq0: } \| 7 + 2i \| = 7$$

$$\text{sorted [1] = in_seq1: } \| 3 + 4i \| = 5$$

$$\text{sorted [2] = in_seq2: } \| 9 + 9i \| = 12$$

$$\text{sorted [3] = in_seq3: } \| 2 + 0i \| = 2$$

Case2: `in_opcode [2] = 1'b1` => Norm 值小於 7 的優先排序

$$\text{sorted [0] = in_seq1: } \| 3 + 4i \| = 5$$

$$\text{sorted [1] = in_seq3: } \| 2 + 0i \| = 2$$

$$\text{sorted [2] = in_seq0: } \| 7 + 2i \| = 7$$

$$\text{sorted [3] = in_seq2: } \| 9 + 9i \| = 12$$

(7 並沒有小於 7)

第五步，依 in_opcode [1:0] 指示，進行複數的乘除加減計算。

運算的對象是排序後的複數，定義為以下：

$$\text{sorted}[0] = A + Bi$$

$$\text{sorted}[1] = C + Di$$

$$\text{sorted}[2] = E + Fi$$

$$\text{sorted}[3] = G + Hi$$

$$\text{in_opcode}[1:0] = 2'b00$$

$$Ans = (E + Fi) * (C + Di)$$

$$Ans_{real} = (E * C) - (F * D)$$

$$Ans_{imag} = (E * D) + (F * C)$$

$$\text{in_opcode}[1:0] = 2'b01$$

$$Ans = (G + Hi) / (A + Bi)$$

$$Ans_{real} = (G * A + H * B) / (A^2 + B^2)$$

$$Ans_{imag} = (H * A - G * B) / (A^2 + B^2)$$

$$\text{in_opcode}[1:0] = 2'b10$$

$$Ans = \log_2(||A + Bi||) + \log_2(||E + Fi||)$$

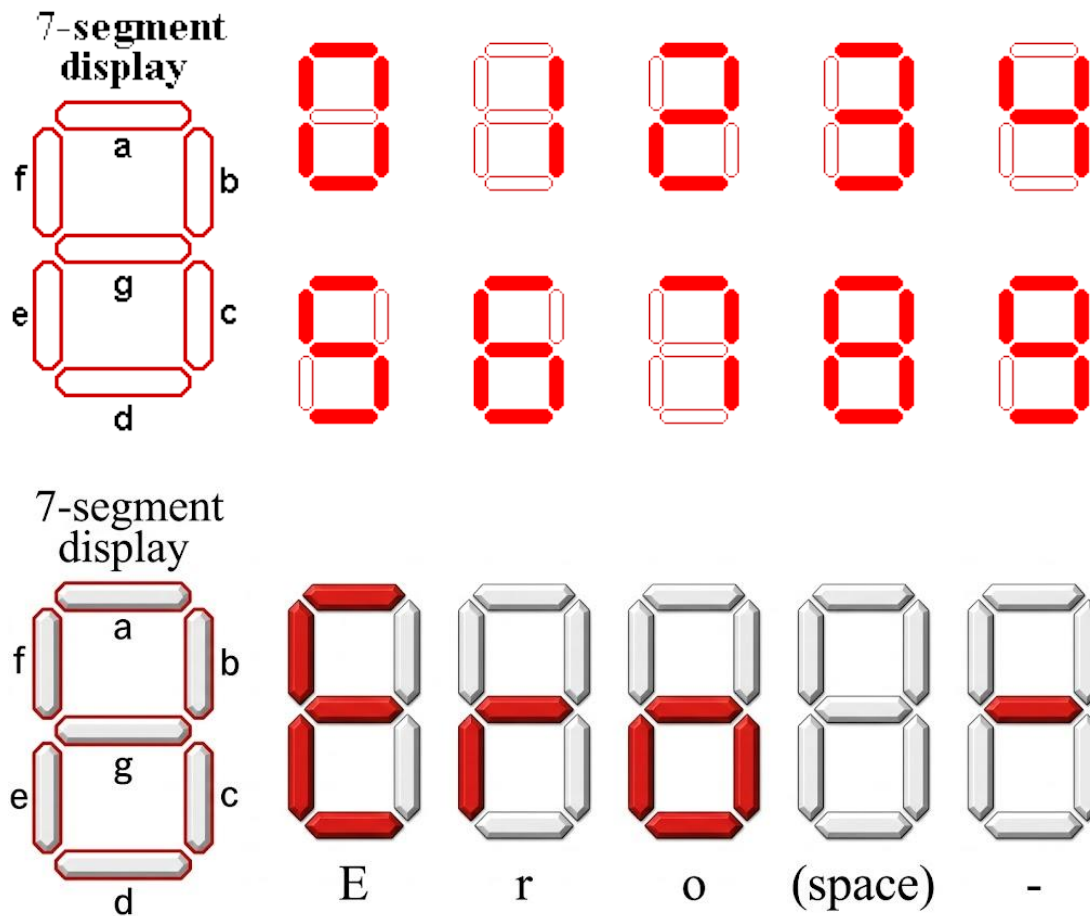
$$Ans_{real} = Ans, \quad Ans_{imag} = 0$$

$$\text{in_opcode}[1:0] = 2'b11$$

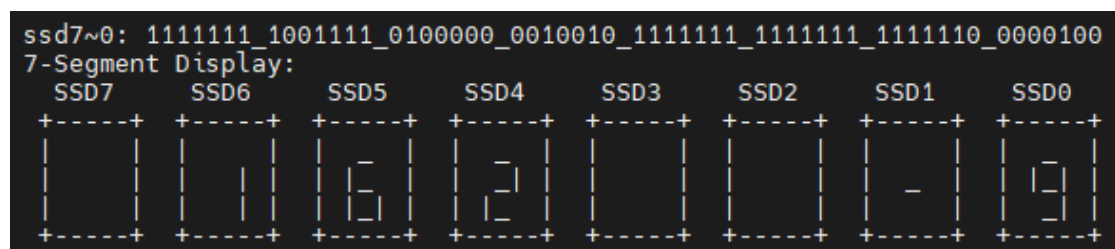
$$Ans = (C + Di)(C + Di)^* - (G + Hi)(G + Hi)^*$$

$$Ans_{real} = Ans, \quad Ans_{imag} = 0$$

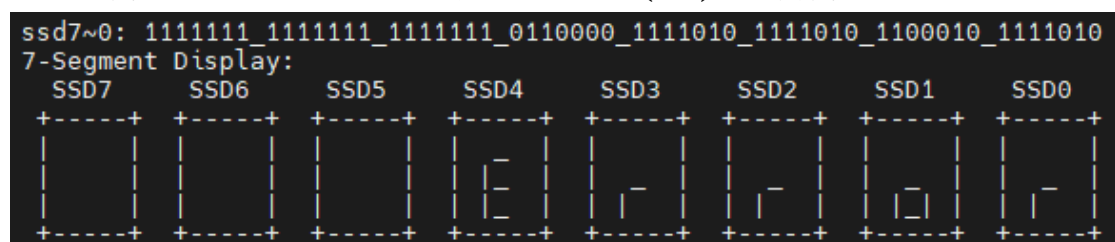
第六步，將計算結果使用七段顯示器輸出 (active-low)。七段顯示器視覺上由左至右分別為 out_ssd7 ~ out_ssd0。訊號 MSB ~ LSB 對應 a, b, c, d, e, f, g。(本作業輸出範圍必定能在七段顯示器成功顯示且不會溢位)



SSD7 ~ SSD4 為實數部分的顯示器；SSD3 ~ SSD0 為虛數部分的顯示器。



若輸入出現非 BCD 表示的數值，或出現 (0, 0)，則需輸出 Error。



Inputs

Signal name	Number of bit	Description
in_opcode	3	Sorting and Arithmetic Instructions
in_seq0	8	Complex Number Sequence
in_seq1	8	Complex Number Sequence
in_seq2	8	Complex Number Sequence
in_seq3	8	Complex Number Sequence

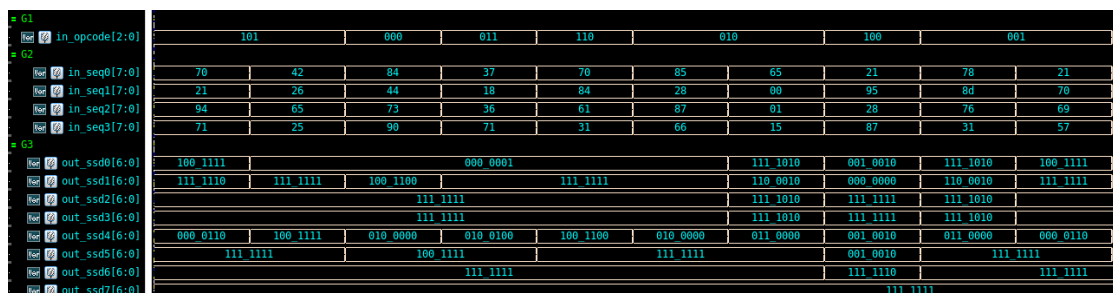
Outputs

Signal name	Number of bit	Description
out_ssd0	7	Seven-Segment Display
out_ssd1	7	Seven-Segment Display
out_ssd2	7	Seven-Segment Display
out_ssd3	7	Seven-Segment Display
out_ssd4	7	Seven-Segment Display
out_ssd5	7	Seven-Segment Display
out_ssd6	7	Seven-Segment Display
out_ssd7	7	Seven-Segment Display

Specifications

1. Top module name: **ComplexCalc** (File name : **ComplexCalc.sv**)
2. 請用 systemverilog 和 combinational circuit 完成你的作業。
3. 02_SYN result 不行有 error 和 latch，且 slack result 必須要 MET。
4. 03_GATE 不行有 timing violation。
5. 請不要使用包含 **error**、latch、congratulation、**fail** 字串的變數名稱。
6. **禁止使用 for 迴圈**。

Example waveform



Debug

1. 在 Terminal 有附 Debug 資訊，包含 Sorted 完的正確結果、計算完的正確答案、和七段顯示器的輸出結果。

```
=====
                                ANSWER FAIL!
                                Pattern No.  2
                                DEBUG INFO  =====
=====
[INPUTS]  in_opcode = 000 (Stable-Sort:0, Calc-Mode:00)
          in_seq0 = 84 (Real:8, Imag:4), in_seq1 = 44 (Real:4, Imag:4)
          in_seq2 = 73 (Real:7, Imag:3), in_seq3 = 90 (Real:9, Imag:0)
=====
[DEBUG]   Sorted seq: A=8, B=4, C=4, D=4, E=7, F=3, G=9, H=0
[DEBUG]   Calculated: Real = 16, Imag = 40
=====
[GOLDEN]  ssd7~0: 1111111_1111111_1001111_0100000_1111111_1111111_1001100_0000001
[GOLDEN]  7-Segment Display:
          SSD7   SSD6   SSD5   SSD4   SSD3   SSD2   SSD1   SSD0
          +---+ +---+ +---+ +---+ +---+ +---+ +---+ +---+
          |   | |   | |   | |   | |   | |   | |   | |   |
          |   | |   | |   | |   | |   | |   | |   | |   |
          |   | |   | |   | |   | |   | |   | |   | |   |
          |   | |   | |   | |   | |   | |   | |   | |   |
          |   | |   | |   | |   | |   | |   | |   | |   |
          |   | |   | |   | |   | |   | |   | |   | |   |
          +---+ +---+ +---+ +---+ +---+ +---+ +---+ +---+
[YOURS ]  ssd7~0: 1111111_1111111_1001111_0000100_1111111_1111111_0000110_0001111
[YOURS ]  7-Segment Display:
          SSD7   SSD6   SSD5   SSD4   SSD3   SSD2   SSD1   SSD0
          +---+ +---+ +---+ +---+ +---+ +---+ +---+ +---+
          |   | |   | |   | |   | |   | |   | |   | |   |
          |   | |   | |   | |   | |   | |   | |   | |   |
          |   | |   | |   | |   | |   | |   | |   | |   |
          |   | |   | |   | |   | |   | |   | |   | |   |
          |   | |   | |   | |   | |   | |   | |   | |   |
          |   | |   | |   | |   | |   | |   | |   | |   |
          |   | |   | |   | |   | |   | |   | |   | |   |
          +---+ +---+ +---+ +---+ +---+ +---+ +---+ +---+
                                at 30000 ns
=====
```

Upload file

1. Code 使用 09_UPLOAD/01_upload 上傳。
2. Report 檔名 **hw1_report_dcsXXX.pdf**, XXX is your server account. 上傳至 E3。
3. Note that the script requires time to synchronize with the NTP server. Please avoid last-minute submissions.
4. 1de 請在 3/19 (四) 23:59 之前上傳 / 2de 請在 3/24 (二) 15:30 之前上傳。

Grading policy

1. Pass the RTL & SYN & GATE simulation. 60%
2. Performance: 02_SYN Area 30%
3. Report 10% (請參考附件格式)
4. Submission in 2nd demo will get 40% off.

Note

Template folders and reference commands:

1. 01_RTL/ (RTL simulation) → **./01_run_vcs_rtl**
2. 02_SYN/ (synthesis) → **./01_run_dc**
3. 03_GATE/ (GATE simulation) → **./01_run_vcs_gate**
4. 09_UPLOAD/ (upload) → **./01_upload**